2022 10th International Conference in Software Engineering Research and Innovation (CONISOFT) | 978-1-6654-6126-9/22/$31.00 ©2022 IEEE | DOI: 10.1109/CONISOFT55708.2022.00022

Process Mining Model Integrated with Control Flow, Case, Organizational and Time Perspectives in a Software Development Project

Silvia Jaqueline Urrea-Contreras, Brenda L. Flores-Rios, Félix Fernando González-Navarro
Universidad Autónoma de Baja California
Instituto de Ingeniería
Baja California, México
{jaqueline.urrea, brenda.flores}@uabc.edu.mx

María Angélica Astorga-Vargas, Jorge Eduardo Ibarra-Esquer
Universidad Autónoma de Baja California
Facultad de Ingeniería
Baja California, México
{angelicaastorga, jorge.ibarra}@uabc.edu.mx

Iván A. García Pacheco, Carla L. Pacheco Agüero
Universidad Tecnológica de la Mixteca
División de Estudios de Posgrado
Oaxaca, México
{ivan, leninca}@mixteco.utm.mx

***Abstract*— This paper presents the mining of a software development project from the event logs stored in the software project management systems Issue Tracking System (Jira) and Version Control System (GitHub) to query commits. In addition, a private system owned by the small software development organization was also included that provides for the event streams stored in Jira and GitHub. The mined project implements the approach to integrated model covering the Process Mining perspectives to validate the execution of activities according to the very small software company's process definition, discover deviations and establish corrective actions based on the findings obtained through the control flow, organizational, case, and time perspectives.**

Keywords-Process mining; Software engineering; Software development projects

I. INTRODUCTION

Process Mining had been applied principally to business processes aims to discover, monitor and improve process models through the extraction of knowledge from the data contained in event logs [1] related to the actual execution of the processes [2], and verify the quality of the discovered models. In general, when getting a process model with process discovery, only its flow of control is shown. To extend these models, some perspectives are added to provide a characterization of the aspects that the mining process intends to analyze [3]. Therefore, in a way that integrates the three types of mining (discovery, conformance checking and improvement) and the Process Mining perspectives can be identified. Process Mining perspectives (control flow, organizational, case and time) offer a description of the aspects of the mining process, enable correlations to find relevant information and may overlap partially [3]. Apply Process Mining into software development can be seen as a tool for the diagnosis, treatment and prevention of software-related problems. Such incorporation is valuable for the following reasons: 1) uses process discovery techniques to capture the actual behavior of the software process, 2) employs conformance checking techniques to detect deviations, 3) predicts performance or conformance related issues from of the alignment of the models and the actual behavior of the software [4, 5]. Additionally, it is convenient use the Process Mining perspectives so that can discover the process from different points of view allowing the organizations to solve specific problems identified in the software processes and minimize their impact.

Different ways of carrying out Process Mining into software processes have been described; as a software process tool, for example, for requirements analysis or to investigate the execution of software systems; another way is the direct application of Process Mining to the software process data [4]. Since data from project management systems (Jira, GitHub, Bitbucket, Redmine, Subversion and others) can be extracted through tools or APIs, this allows researchers to collect available information from those software project management systems for further analysis. Nevertheless, how to automatically extract a process model from the massive process data generated by existing software development organizations has become a focus of current software process research [5].

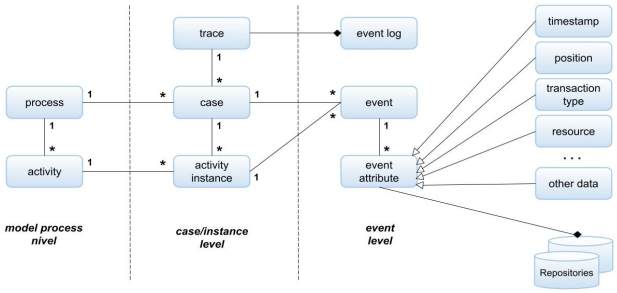

Figure 1. Representation of event log elements (adapted from [3]).

Authorized licensed use limited to: Pontifica Universidade Catolica do Parana. Downloaded on April 12,2026 at 23:15:57 UTC from IEEE Xplore. Restrictions apply.

Due to software processes are generally not well structured compared to business processes, with considerable variation between different instances of the same process [4]. Software process data is generated by activities such as software development, evolution, maintenance, and testing [6]. The data generated from such activities can be extracted from the management systems to formulate a software process event log. However, if the process is not all automated or the management systems have not connected, the activities of the software process are left outside the register event log. A software process event log is a two-dimensional table that is used in Process Mining to describe the information in the event [5]. As the event log is the starting point of Process Mining [3], data from software processes can be handled according to the elements that define the event log (Fig. 1) and therefore provide the input for applying Process Mining techniques.

This paper focuses on the application of Process Mining over a software development process within a real very small software company implementing the approach to integrated model covering the control flow, organizational, time, and case perspectives presented in [3]. The objectives of this study are O1) to obtain an integrated model of processes related to a software development project in such a way that the company can know if the execution of the processes are consistent with their documented process, discover if the execution of the processes are consistent with their documented process, O2) discover if there are deviations within its model through identify non-conforming activities. So based on the detected findings, the company could establish corrective actions to make its processes more efficient.

The document is structured as follows: section II presents the background related to Process Mining in software development projects; section III details the approach to integrated model covering the control flow, organizational, time, and case perspectives; section IV expounds the implementation of the methodology with the characterization of the software development organization, obtaining the event log, the description of the discovered process model and the verification of the events within their activities; section V shows the results of the analysis with the Process Mining perspectives. Section VI presents a discussion. Finally, the conclusions and future work are presented in section VII.

II. Related Work

In [7] a systematic literature review was conducted to specify the transdisciplinarity of Process Mining perspectives (control flow, organizational, case, and time) in software engineering taking into account the topics of knowledge areas (Software Engineering Education Knowledge - SWEEK) [8]. The result presented the contributions in the area of metrics and management, design tools and techniques, software process management, and Software Engineering Professional Practice. Thus, the findings indicated that Process Mining did not focus on any specific stage of software development and identified that the control flow and organizational perspectives are the most implemented in comparison to the case and time perspective [7, 9].

In [10] the authors analyzed a dataset of bug fix processes for 30 Apache open-source projects that use Jira as an issue tracking system to report bugs. The study proposed an Anomaly Detection Framework that includes four phases: 1) data extraction, 2) data cleaning and transformation, 3) anomaly detection, and 4) performance evaluation. The results presented a formalized approach for pre-processing and transformation of crash report history data and a Process Mining-based Anomaly Detection Framework for bug fix processes comprising algorithms proposed for process discovery and conformance checking. As future work, the implementation of multi-perspective anomaly detection was established.

Similarly, in the work presented by [11], control flow, case, performance and organizational perspectives were applied. Data were extracted from two software projects stored in Jira of an Information Technology company that follow the Scrum methodology. A case study was established for the analysis of the projects where the application of Process Mining (PM^2 methodology [12]) in management systems and Scrum processes was evidenced. In addition, some challenges were identified in relation to the application of Process Mining to study software development processes from software management systems data. This brings up the lessons learned from the case study that are useful when doing similar analysis on other organizations and projects, especially (but not limited to) those that depend on the Jira System.

On the other hand, in [13] proposed a Process Mining methodology for the analysis of software project development involving the Git log pre-processing and the manual analysis of discovered model based on the analysis of Git logs from student projects in order to understand their software development processes. The results were analyzed through the control flow, performance and time perspectives. While in, Kalliamvakou et al. [14] data is analyzed from GitHub to understand the characteristics of the repositories and their users, as well as to evaluate if they are suitable as a data source for software engineering studies. The results presented a set of recommendations on how to approach the GitHub data, highlighting the need to, first, evaluate data adequacy so that, then, the data that can really provide information to answer the research questions can be emphasized.

In related works such as [15] and [16], conformance checking has been carried out on software projects. For example, the coding behaviors are verified from the event logs generated by the use of IDEs [15], whereas in [16], a comparison was made between the events corresponding to an activity developed by different software development teams on an IDE. The event logs obtained were compared against a process model indicated as best practice to measure them in terms of fitness and precision and

Authorized licensed use limited to: Pontifica Universidade Catolica do Parana. Downloaded on April 12,2026 at 23:15:57 UTC from IEEE Xplore. Restrictions apply.

understand their real software development process. In addition, [15] and [16] uses a series of stages similar to Process Mining methodologies such as PM^2 [12] and the L* Life-Cycle Model [3] where a detailed data preparation is performed for further analysis.

None of these studies were focused on applying Process Mining in a very small software company, even though their projects are typically small-sized with limited resources and staff, the results of Process Mining can benefit them to improve their processes. Therefore, based on the objectives of this study to discover different points of view of the software process in the real context of a very small software company and the scope of the methodologies analyzed in the case studies to mining processes, the approach integrated model covering the control flow, organizational, time, and case perspectives was selected.

III. METHODOLOGY

Fig. 2 shows an adapted graphical representation of the approach integrated model covering the control flow, organizational, time, and case perspectives, that comprises the following phases: 1) obtain event logs from management systems (repositories); 2) create/describe the model; 3) verify events and activities, regarding their conformance checking; 4) expand the model based on the three perspectives (organizational, case and time), and 5) obtain an integrated model with the representation of all perspectives. This adaptation consists in presenting explicitly the inputs and outputs of the work products according to the best practices of software engineering.

IV. IMPLEMENTATION

A. *Characterization of the software development company*

The participating company, dedicated to software development, is established in Baja California, Mexico, with a portfolio of projects at regional, national and international levels (Australia, United States). It is a 10-year-old company with 15 employees. The company has achieved an effective implementation of Software Process Improvement with the CMMI-DEV v1.3, for which in 2019 it obtained the level 2 certification.

1) Company software development process: The specific methodologies for software development and maintenance adopted by the company are Scrum [17] and Kanban [18]. The company's software development process begins when the Project Department gives the Software Architect the proposed general features of the software to carry out the Requirements Analysis, in which a prototype is also designed, and then validated by the client. Once validated, the Project Manager defines the user stories based on the features and the prototype. Then, the Software Architect establishes the Solution Diagram that integrates the data model and the technologies to be used are defined. Kanban Development is managed through issues, with the features of the software to be developed and the technical work. The following step is Software development, when a Programmer performs code protection, which proceeds to Continuous Integration (CI) and Continuous Deployment (CD). In CI, the code protection is compiled to carry out automatic tests and then its packing and storage. DC contemplates a controlled testing and staging environment where changes are validated with real process information. Finally, the production environment with the final software product gets validated by the client.

2) Project Description and Roles: The project under study corresponds to a Web application for gym services management that includes the functions of: customer registration, membership payment, fee renewal and access control. This project lasted approximately 2 months (May - June 2021). The project team was made up of the roles of: customer registration, membership payment, fee renewal and access control. This project lasted approximately 2 months (May - June 2021). The project team was made up of the roles of: Project Manager, Technical Leader, Programmer and Quality Manager. The Project Manager, as well as the Technical Leader, have been with the company for 10 years. The role of Programmer was performed by 2 members of the team: a junior programmer with 2 years of experience working in the company and dedicated full-time

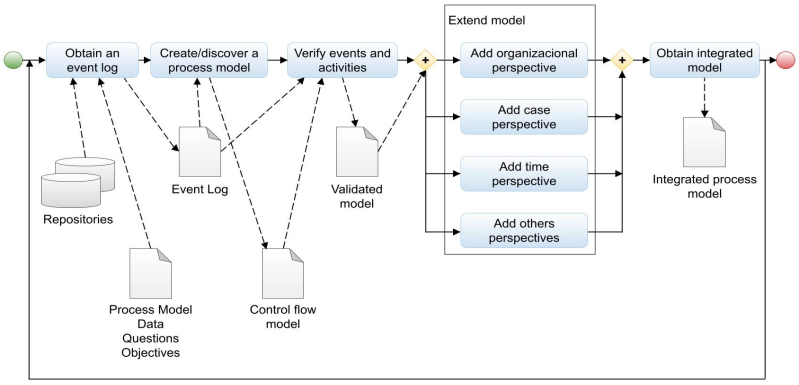

Figure 2. Graphical representation of the approach integrated model covering the control flow, organizational, time, and case perspectives from [3].

Authorized licensed use limited to: Pontifica Universidade Catolica do Parana. Downloaded on April 12,2026 at 23:15:57 UTC from IEEE Xplore. Restrictions apply.

to the project, and a second, medium level programmer with 6 years of experience, who was assigned the most complex issues and at the same time participated in another software development project. According to what was stated by the Project Manager, the project began with the intervention of two programmers but ended with a single programmer due to the workload and the economic situation of the company stemming from the COVID-19 pandemic. Also, at the time of finishing this project, the Quality Manager, with 5 years of seniority, stopped working in the company.

3) Repositories: The technological tools used by the company to store the event logs in the software development process are: Jira, Github and SIMIo. Each of them is briefly described below.

a) JIRA: Jira can integrate applications into a tightly integrated product development solution for teams that need to do more in less time [19]. For example, the Time in Status application can generate reports related to the time that a status has been maintained within the issue, its average duration, date of entry to a particular status, transitions and status count. On the other hand, with the Issue History for Jira application [20] one can monitor the changes of the selected issues, keep track of all the updates that have been made to an issue, choose the keys of the issues whose changes must be monitored, and sort them by date and key. In Jira the development process begins by creating a Kanban board set to the following flow, which is triggered by status changes: Selected for Development, To Do, In Progress, Verification, Quality Assurance, Rejected, Validation, and Done. In this way, issues are tracked (Issue Tracking System). The requirements are found in user stories, which are planned by sprints and distributed among the project team in a self-organized way in the execution of their tasks. The constant updating of the system information and its availability to all the project members allows an analysis of the follow-up of the issues present in a software project. The automatic update of the information allows the export of a complete history using filters or sentences that Jira provides, as well as datasets according to the different flows that the process can follow.

b) GITHUB: Once the tasks are in the To Do status, the version control flow continues through the implementation of an interface between Jira and Version Control System GitHub [21] from the existing issues at the moment of passing from To Do to In Progress status, when the creation of a branch or a commit is triggered. The interface is related by the company with the purpose of maintaining traceability between project planning and code management in terms of code writing collaboration between the different programmers, testing and deploying the versions that are generated where they can add annotations and discuss certain changes made to a specific commit.

c) SIMIo: It is a software tool developed by the company to keep a detailed record of the work done by the project team members. This software completes the event stream stored in Jira and GitHub. In the gymnasium project, only the activities and hours spent working by the programmers were recorded according to the scope of the SIMIo version at that time. From SIMIo can download a report with the name of the project, the name of the team member registering the work, their role, date, details of the work, hours spent working and effective hours. The company defined a formula to calculate the effective hours of the project that, together with the report, provide significant information for decision-making by the Project Manager and Senior Management.

B. Obtaining and creating the Event Log

In the first phase of the approach integrated model, a first extraction of the events recorded by Jira and SIMIo was carried out. The history of status and changes in the flow of activities (issues) documented in the Kanban board were obtained from the Jira repository, while the extended description of the activities registered by the programmers was obtained from the SIMIo repository. The relationship of the extracted event logs in terms of their traceability with SIMIo was done manually due to the absence of an explicit attribute for their relationship with Jira. The manual traceability was carried out in collaboration with the Project Manager, which consisted of reviewing the events obtained from SIMIo to relate them to the Jira issue key to which they referred. Based on this traceability, inconsistencies in some event logs were corrected and software development stages were assigned (Planning, Development, Testing and Deployment).

Once the traceability between the repositories was performed, the event log was made up of the cardinalities and relationships of the elements present in Fig. 1. Thus, from the repositories and the data elements they stored, the following event-describing attributes were defined: 1) Issue Key as case identifier, 2) Status of the issue as a way of defining the activities, 3) Timestamp (date and time) specifying the execution of the activity, 4) Role that performs the activity, 5) Actor who performs the activity, and 6) Software development stage to which the activity belongs. The first five attributes belong to the Jira registry and the last one (Software development stage) corresponds to the manual traceability between SIMIo and Jira.

Each log event refers to an activity; 10 activities were obtained that are executed in the development process in the following order: 1) Create Issue, 2) Backlog, 3) Selected for Development, 4) To Do, 5) In Progress, 6) Verification, 7) Quality Assurance, 8) Rejected, 9) Validation, and 10) Done. Table I shows the activities in the software process event log. The Create Issue activity is the one that starts each event and the Done activity corresponds to the end activity. The Done activities have a frequency of 65, while the Create Issue activities have a frequency of 62. It was observed that this is because the issues were re-opened after assigning them as Done to finish them again.

Authorized licensed use limited to: Pontifica Universidade Catolica do Parana. Downloaded on April 12,2026 at 23:15:57 UTC from IEEE Xplore. Restrictions apply.

TABLE I. ACTIVITIES IN THE SOFTWARE PROCESS EVENT LOG

Order	*Activity*	*Frequency*
1	Create Issue	62
2	Backlog	27
3	Selected For Development	43
4	To Do	40
5	In Progress	54
6	Verification	62
7	Quality Assurance	63
8	Rejected	1
9	Validation	64
10	Done	65
	TOTAL	481

Each instance of an activity is related to a case (Fig. 1); for this reason, the issue status was selected as an activity to be able to establish the cases and that these were based on the issue key. As shown in Fig. 3, the event log fragment represents the events that make up the case of issue 8, where each column and row correspond to an attribute and event, respectively. For each case, an event trace can be recorded that corresponds to the execution of the case from start to finish [3]. In this way, the event log traces marked the possible paths through the activity flow of the process model.

With the above, the initial event log showed a total of 488 events distributed in 62 cases, which, after debugging a total of 481 events was obtained without the elimination of a case. The 100% of the evidence of the execution of the project activities was covered. The debug was due to a duplicity of the activity state and the execution time (timestamp). That is, there was no change of state. For example, from In Progress to In Progress, To Do to To Do and Done to Done.

The extraction of event logs from the two software repositories, Jira and SIMIo, as well as their manual traceability, allowed to obtain important insights about the control flow of the software development process. The challenge of obtaining control flow was found in the identification of cases and activities.

C. *Creation and discovery of the model*

In the second phase, Create or discover the model, inductive mining was applied with the purpose of generating the real process model in a visual format represented by Petri nets. This model was obtained with the support of the Mine Petri net with Inductive Miner plugin of the ProM tool through the analysis of the event logs. Fig. 4 displays the model discovered through a petri net and its BPMN diagram with the different flows that the transition status of the issues presented according to the event log.

ATTRIBUTES

EVENT LOG

Issue ID	Status	Timestamp	Role	Actor	Stage
BPF-8	CREATE ISSUE	5/3/2021 16:44	Project manager	Actor A	Planning
BPF-8	BACKLOG	5/3/2021 16:44	System	System	Planning
BPF-8	SELECTED FOR DEVELOPMENT	5/4/2021 13:09	Project manager	Actor A	Planning
BPF-8	TO DO	5/4/2021 13:12	Project manager	Actor A	Planning
BPF-8	IN PROGRESS	5/11/2021 15:46	Programmer	Actor C	Development
BPF-8	VERIFICATION	5/14/2021 13:05	Programmer	Actor C	Development
BPF-8	QUALITY ASSURANCE	5/17/2021 9:50	Quality manager	Actor E	Testing
BPF-8	VALIDATION	5/20/2021 12:45	Quality manager	Actor E	Testing
BPF-8	DONE	5/21/2021 9:53	Project manager	Actor A	Deployment

Start

T R A C E

End

CASE

Figure 3. Event log snippet from the software development process.

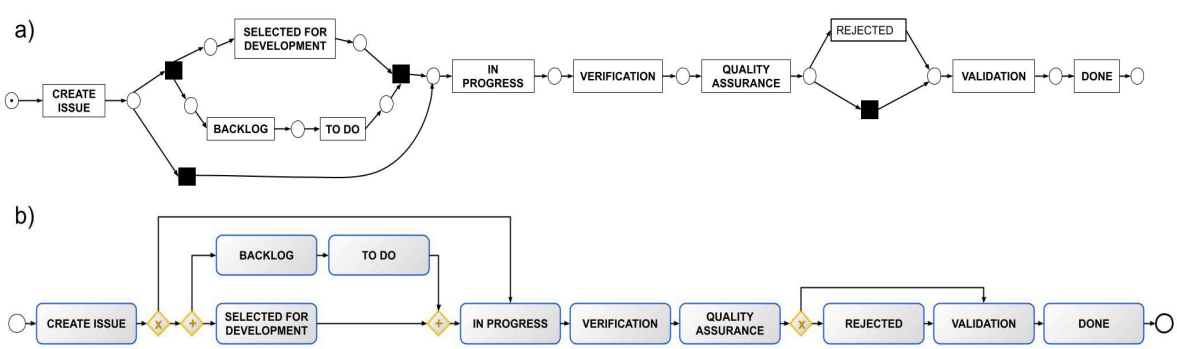

Figure 4. Real process model of issue transition status in Jira, a) Petri net, b) BPMN diagram.

Authorized licensed use limited to: Pontifica Universidade Catolica do Parana. Downloaded on April 12,2026 at 23:15:57 UTC from IEEE Xplore. Restrictions apply.

Subsequently, the traces were analyzed based on their similarity, according to the profiling approach, which uses the characteristics derived from the event information of each trace or the grouping of traces, in order to discover models of simple processes which, together, represent a complex process model [22].

From the review of the event log, it was observed that the trace with the shortest length is made up of 3 events and the longest has 13 events, while 56.45% of the traces are composed of 6 or 9 events (Fig. 5). In the histogram of trace events in Fig. 5, the groups formed by the traces with a close similarity are presented by color. It was identified that the most frequent trace is related to 20 cases, which represents 32.26% of the event log. This trace is made up of the activities σ_1= 〈 Create Issue, In Progress, Verification, Quality Assurance, Validation, Done〉.

Table II shows the 22 traces identified in the event log; it is observed that the trace with the greatest presence corresponds to defect solving (error-type Issue) with 32.26%, which represents a little less than a third of the total event log. On the other hand, the second most frequent trace corresponds to development issues (Feature) with 24.19%, which represents a quarter of the total event log. The other traces are slight tracking changes between the traces with greater frequency; for example, for the error-type Issue traces, the omission or integration of the In Progress, Quality Assurance and Rejected status can be seen. Likewise, the Error traces is very similar to the Rework traces, where the status of Selected for Development and To Do are integrated. Regarding the Development trace (Feature), the changes to originate new traces were due to the repetitive change of the issue between the Selected for Development and To Do status.

D. Verification of events and activities

In the third phase, conformance checking was implemented applying the Token Replay technique in the real process model (Fig. 4). Token Replay measures fitness at the event level (trace events that can be reproduced in the model) providing a result between 0-1 where 1 means perfect fitness. A model has perfect fitness if all traces in the log can be reproduced by the model from start to finish [3, 23, 24]. Considering the relationship of the events in the event log with the activities of the process model discovered in Phase 2, a general fitness of 0.935 was obtained, that is, around 6.5% of the events show deviations. Likewise, 61.3% of the traces in the log are characterized by a fitness level of 1.

Figure 5. Fragment of the event log of the software development process.

TABLE II. SET OF TOTAL TRACES PRESENT IN THE EVENT LOG. A= CREATE ISSUE, B= BACKLOG, C= SELECTED FOR DEVELOPMENT, D= TO DO, E= IN PROGRESS, F= VERIFICATION, G= QUALITY ASSURANCE, H= REJECTED, I= VALIDATION AND J= DONE

Issue Type	*Trace*	*Cases*
Error	σ_1= 〈 *a, e, f, g, i, j* 〉	20
Feature	σ_2= 〈 *a, b, c, d, e, f, g, i, j* 〉	15
Error	σ_3= 〈 *a, f, g, i, j* 〉	5
Feature	σ_4= 〈 *a, b, c, d, c, d, c, d, e, f, g, i, j* 〉	2
Error	σ_5= 〈 *a, i, j* 〉	2
Error	σ_6= 〈 *a, e, f, g, j, i, j* 〉	2
Feature	σ_7= 〈 *a, b, c, d, e, f, g, f, g, i, g, i, j* 〉	1
Feature	σ_8= 〈 *a, b, c, d, c, d, e, c, d, f, g, i, j* 〉	1
Feature	σ_9= 〈 *a, b, c, d, c, d, f, g, i, j* 〉	1
Feature	σ_{10}= 〈 *a, b, c, d, c, d, e, f, g, i, g, i, j* 〉	1
Feature	σ_{11}= 〈 *a, b, c, d, c, d, e, f, g, i, j* 〉	1
Technical Labor	σ_{12}= 〈 *a, c, b, c, d, e, f, g, i, j* 〉	1
Feature	σ_{13}= 〈 *a, b, c, d, c, d, c, g, i, j* 〉	1
Feature	σ_{14}= 〈 *a, b, c, d, c, f, d, e, g, i, j* 〉	1
Feature	σ_{15}= 〈 *a, b, c, d, e, f, g, f, g, i, j* 〉	1
Error	σ_{16}= 〈 *a, e, f, i, j* 〉	1
Rework	σ_{17}= 〈 *a, c, e, f, g, i, j* 〉	1
Rework	σ_{18}= 〈 *a, c, d, e, f, g, i, j* 〉	1
Feature	σ_{19}= 〈 *a, b, c, d, e, f, e, f, g, i, j* 〉	1
Technical Labor	σ_{20}= 〈 *a, c, d, e, f, g, j* 〉	1
Error	σ_{21}= 〈 *a, e, f, g, i, j, i, j* 〉	1
Error	σ_{22}= 〈 *a, e, f, g, h, i, j* 〉	1
	TOTAL	62

Table III shows the resulting fitness for the rest of the traces in the log, followed by the alignments of the missing activities, so that the trace can be reproduced at an optimal level, and the wrong activities, which are those activities that present a double reproduction in the trace while the model does not allow it, this is the reason for which the movement of their transition is performed at the log level and not in the model.

The missing activities were b= Backlog, e= In Progress, f= Verification, g= Quality Assurance and i= Validation, representing their conformance checking in the model and in Table IV with their overall fitness percentage. The results of Table IV indicated that the Backlog activity has a fitness of 93.1%, In Progress 85.5%, Verification 93.5%, Quality Assurance 95.2% and Validation 98.4%. This indicates that the Validation activity shows deviations in 1.6% of its events, which is reflected in the missing column of trace 20 as Validation is mentioned only once in the column.

Authorized licensed use limited to: Pontifica Universidade Catolica do Parana. Downloaded on April 12,2026 at 23:15:57 UTC from IEEE Xplore. Restrictions apply.

TABLE III. FITNESS AND ALIGNMENTS OF THE WHOLE SET OF TRACES PRESENT IN THE EVENT LOG

Trace	*Fitness %*	*Missing*	*Wrong*
σ_1= ⟨ *a, e, f, g, i, j* ⟩	100	-	-
σ_2= ⟨ *a, b, c, d, e, f, g, i, j* ⟩	100	-	-
σ_3= ⟨ *a, **e***, f, g, i, j* ⟩	92.59	*e**	-
σ_4= ⟨ *a, b, c, d, **c**, **d**, **c**, **d**, e, f, g, i, j* ⟩	76.47	-	*c, d*
σ_5= ⟨ *a, **e***, **f***, **g***, i, j* ⟩	71.43	*e*, f*, g**	-
σ_6= ⟨ *a, e, f, g, **j**, i, j* ⟩	90.91	-	*j*
σ_7= ⟨ *a, b, c, d, e, f, g, **f**, **g**, **i**, g, i, j* ⟩	76.47	-	*f, g, i*
σ_8= ⟨ *a, b, c, d, **c**, **d**, e, **c**, **d**, f, g, i, j* ⟩	76.47	-	*c, d*
σ_9= ⟨ *a, b, c, d, **c**, **d**, **e***, f, g, i, j* ⟩	80.95	*e**	*c, d*
σ_{10}= ⟨ *a, b, c, d, **c**, **d**, e, f, g, i, **g**, **i**, j* ⟩	76.47	-	*c, d, g, i*
σ_{11}= ⟨ *a, b, c, d, **c**, **d**, e, f, g, i, j* ⟩	86.67	-	*c, d*
σ_{12}= ⟨ *a, c, b, **c**, d, e, f, g, i, j* ⟩	98.86	-	*c*
σ_{13}= ⟨ *a, b, c, d, **c**, **d**, **c**, **e***, **f***, g, i, j* ⟩	69.05	*e*, f**	*c, d*
σ_{14}= ⟨ *a, b, c, d, **c**, **f**, **d**, e, **f***, g, i, j* ⟩	75.56	*f**	*c, d, f*
σ_{15}= ⟨ *a, b, c, d, e, f, g, **f**, **g**, i, j* ⟩	86.67	-	*f, g*
σ_{16}= ⟨ *a, e, f, **g***, i, j* ⟩	92.59	*g**	-
σ_{17}= ⟨ *a, **c**, e, f, g, i, j* ⟩	90.91	-	*c*
σ_{18}= ⟨ *a, **b***, c, d, e, f, g, i, j* ⟩	94.44	*b**	-
σ_{19}= ⟨ *a, b, c, d, e, f, **e**, **f**, g, i, j* ⟩	86.67	-	*e, f*
σ_{20}= ⟨ *a, **b***, c, d, e, f, g, **i***, j* ⟩	87.88	*b*, i**	-
σ_{21}= ⟨ *a, e, f, g, i, j, **i**, **j*** ⟩	83.33	-	*i, j*
σ_{22}= ⟨ *a, e, f, g, h, i, j* ⟩	100	-	-

TABLE IV. PERCENTAGE OF CONFORMANCE CHECKING WITH RESPECT TO ACTIVITY

Status/Activity	*Fitness*
Backlog (b)	93.1%
In Progress (e)	85.5%
Verification (f)	93.5%
Quality Assurance (g)	95.2%
Validation (i)	98.4%

Obtaining a fitness of 93.5% with the conformance checking, and representing around 450 reproducible events in the discovered process model, it is concluded that the model is validated for the application of the Process Mining perspectives according to [3, 25].

V. RESULTS

A. Control Flow Perspective

In the control flow perspective, we can observe the order of the activities within the extracted event log that are represented by issue status. As a first activity, an association analysis was carried out with the stages of the software development life cycle and the activities included in the event log (Table V). Fig. 6 shows that 3 iterations were executed in the project.

ProM Multi-perspective Process Explorer presents the resulting process model in which one can visualize that the activities with the highest execution rate are Create Issue, In Progress, Verification, Quality Assurance, Validation and Done, which correspond to the trace with the greatest frequency within the event log (Fig. 7). Also, the least executed activity is Rejected, which only ran once in the event log. In Fig. 8, the incorporation of the flows can be visualized so that the model obtains a full conformance checking (100%). The incorporated flows are based on trace identification and the transitions show the number of times the flow was used.

B. Case Perspective

For the analysis of the case perspective, each issue was established as a case (trace), from which it was obtained that the trace with the highest frequency in the event log corresponds to the issues labeled as Error, which in most cases go to a state of In Progress after their creation to resolve the issue. Similarly, the second trace with the highest presence was obtained (15 traces representing 24.19% of the event log) corresponding to the issues labeled as Feature. Features entail a process that more clearly details the stages of software development. If the complete development process described by the company is taken, it follows that the logs present in the repositories of section IV detail information from the Kanban development until a deployment is made in the production environment.

C. Organizational Perspective

Organizational mining can be used to obtain information about typical work patterns, organizational structures, and social networks. It is based on the resource attribute that can contain information about machines, roles, departments and resource distribution [3, 26, 27, 28].

Figure 6. Control flow model of the software development life cycle stages.

TABLE V. ASSOCIATION OF THE KANBAN BOARD DEVELOPMENT FLOW WITH THE SOFTWARE DEVELOPMENT PROCESS

Status/Activity	*Stage*
Create Issue	Planning
Backlog	Planning
To Do	Planning
Selected For Development	Planning
In Progress	Development
Verification	Development
Quality Assurance	Testing
Validation	Testing
Rejected	Testing
Done	Deployment

Authorized licensed use limited to: Pontifica Universidade Catolica do Parana. Downloaded on April 12,2026 at 23:15:57 UTC from IEEE Xplore. Restrictions apply.

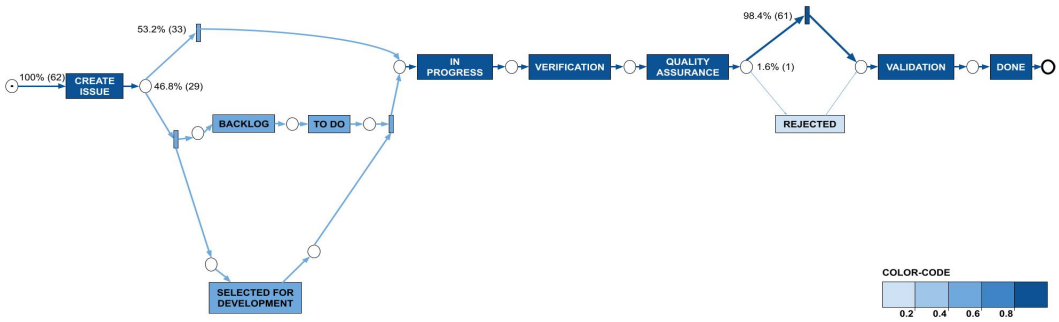

Figure 7. Control Flow Model of the software development life cycle stages.

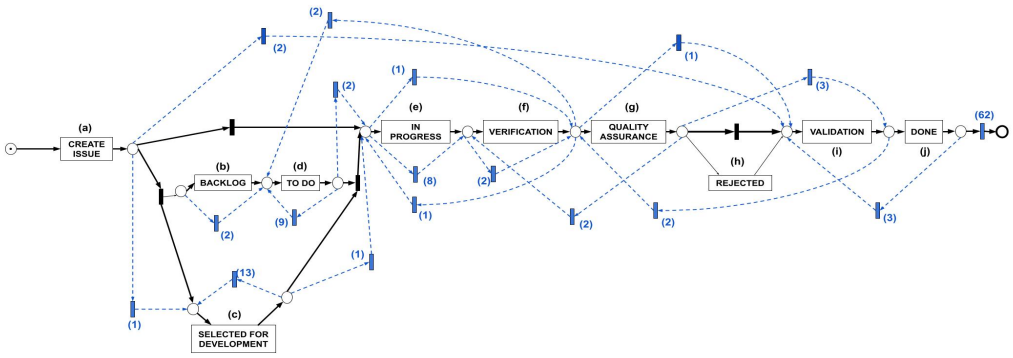

Figure 8. Control Flow Model (100% Conformance)

The organizational perspective relates to the roles that are present in the event log. For its analysis, the Disco tool was used, which allowed the visualization of both the roles and the actors. For confidentiality purposes, only what is related to the roles will be shown. Disco shows the relative frequency of the roles with respect to the trace of highest frequency, within which the programmer role is indicated as having the highest with 50%, followed by the quality manager with 32.29%. For this trace, there is the system role, whose activities are automatic, with 12.5% then there is the Project Manager with 3.12% and finally the technical leader with 2.08%.

The results of a greater participation by the programmers are due to the fact that the information was extracted from the repositories in which the activities carried out by them are logged. An example of this is the log generated by SIMIo, which only provided information from the programmers due to a limitation in its version; because of this, it was suggested that, for subsequent projects, it would be linked with Jira to log the activities of all those involved in the project.

D. Time Perspective

The time perspective corresponds to the measurement of time obtained by the analysis of the timestamp attribute. The timestamp elements of the log correspond to MM/DD/YYYY HH:MM, having a precision of minutes. The average time, the minimum time and the maximum time of the activities were obtained as shown in Table VI. The average time between Create Issue and Backlog is established as 4.4 seconds, which happens because Backlog is an automatic activity carried out by the system after the creation of an issue. Likewise, Backlog obtained a maximum time of 1 minute and a minimum of 0 was taken because the timestamp format does not consider seconds, causing activities with the same timestamp as their predecessor state to have their duration time reflected as 0. For example, in the BPF-8 issue, the change of status between Create Issue and Backlog was made within the same minute (3/5/2021 16:44), unlike the BPF-20 issue, which was made one minute after the status change Create Issue (5/4/2021 13:10) – Backlog (5/4/2021 13:11). The same happens in activities such as In Progress, Verification and Done, whose status is changed by an action of a role.

Table VI shows that the activities with the longest duration are Selected For Development, To Do and Done. The first two correspond to the fact that waiting for the transition to the next stage is linked to the times established in the project and its iterations. The Done status remains until the customer approves the deliverable, after which it is sent to production. On the other hand, the activity of In Progress shows an average duration of 1.3 days compared to the time of 8.7 days in the To Do state.

Authorized licensed use limited to: Pontifica Universidade Catolica do Parana. Downloaded on April 12,2026 at 23:15:57 UTC from IEEE Xplore. Restrictions apply.

TABLE VI. SPECIFICATION OF THE AVERAGE, MAXIMUM AND MINIMUM TIME PER ACTIVITY

Status/Activity	Time		
	Average	*Maximum*	*Minimum*
Create Issue	-	-	-
Backlog	4.4 s	1 m	-
Selected For Development	5 d	16.7 d	1 m
To Do	8.7 d	21.7 d	1 m
In Progress	1.3 d	7.1 d	-
Verification	1.1 d	10.9 d	-
Quality Assurance	10.8 h	4.9 d	6 m
Rejected	18 h	18 h	18 h
Validation	1.9 d	8.8 d	2 m
Done	8.6 d	22 d	-

VI. DISCUSSION

According with O1, the discovered process model, coincides with the execution flow of the process documented by the company. Likewise, the discovery of 22 traces, which were replicated in the process model discovered, is highlighted. In this way according to O2 seeing discrepancies and the alignments required for the trace to be considered fully optimal (100%). As shown in Table III, trace 13 (σ13) shows a lower fitness (69.05%) compared to the other traces of the log. Its low conformance is due to the omission of the execution of the In progress and Verification status as indicated by the alignment of the trace, as well as the incorrect execution of the Selected for Development and To Do status, which were found to be duplicates.

Trace σ_{13} refers to a single feature-type issue related to "Payment History" for which no evidence was found of the transition of the status corresponding to the development stage (Table V) between Jira and SIMIo; however, on Github there is a reference to a change made without using the issue identifier. A similar case occurs with trace 5 (σ_5): its conformance is 71.43% due to the omission of the In Progress, Verification and Quality Assurance status through which most of the error-type issues occur.

Trace σ_5 turns out to be the shortest (three events) that refers to two error-type issues, none of which shows a reference to the issue within Github. The event log contains five more traces (σ_4, σ_7, σ_8, σ_{10}, and σ_{14}) representing six issues with low conformance of less than 80% due to duplicity/iteration of the Selected for Development, To Do, Verification, Quality Assurance, Validation, and Done status that cannot be replicated in the discovered process model. In the event that the model established flows contemplating said iterations, the conformance checking would not be affected.

Together with the control flow perspective and the case perspective, the main path followed by the error type issue σ_1 and the feature type issue σ_2 was visualized. The other traces correspond to variations of the previous traces, which differ mainly due to the iteration of the status involved with a conformance of less than 80%. In [11] a similar situation arose in the analysis of Jira issues, in which iterations of four activities were submitted before the issue was closed.

On the other hand, the organizational perspective showed that the assignment of the workload was focused on the role of programmer, performed by two actors. The medium programmer participated in the first stage of the project developing the features with the greatest estimation in the user stories, while the junior programmer was assigned the smallest ones.

The time perspective was analyzed from the times presented in Table VI. The status with the longest duration corresponds to Backlog → To Do and Validation → Done with an average duration of 8.7 and 8.6 days, respectively; although the durations are a slightly longer than the other status, they do not add an excess duration that puts the project at risk or represents a bottleneck. The latter takes as a reference the finding of [11] where bottlenecks were identified in status such as Open → Waiting Approval and Open → Resolved with an average duration of 13.5 weeks (three months) and 73.4 days (two and a half months), respectively. Likewise, the main bottleneck is the Resolved → Closed transition, with an average of 35.8 weeks (more than eight months).

VII. CONCLUSIONS AND FUTURE WORK

In this study, we have presented the analysis of a software development project applying the approach integrated model covering the control flow, organizational, time, and case perspectives, which allowed to verify the implementation of the discovered model with the real event log obtained from unlinked repositories.

The analysis indicated that the development team is aware of the process and the support tools required. The discrepancies can reveal undesirable deviations, which would indicate the need for better control of the process or, otherwise, desirable deviations. These would indicate that the process is flexible to face situations not considered in the model. It would be up to the development team to consider whether the deviations are due to the flexibility of the process or due to execution errors. In this way, findings obtained from the analysis of the perspectives can support the Quality and Executive Managers towards continuous improvement and/or decision making.

Regarding the application of Process Mining, it was identified that the main limitation for Process Mining is that the repositories of the organization did not communicate with each other; this represents a difficulty in integrating the elements of the event log as they present in the execution of the process. Likewise, it implies pre-processing with human intervention. In the case of this study, the design of the company's private repository was not intended to establish a relationship that would allow

Authorized licensed use limited to: Pontifica Universidade Catolica do Parana. Downloaded on April 12,2026 at 23:15:57 UTC from IEEE Xplore. Restrictions apply.

the automatic end-to-end exploration of the process and achieve complete mining. As a result, activities such as requirements elicitation, architecture, and design are recorded only in the Kanban board and their execution is not supported in a repository.

As future work, the generation of an integrated model through a multi-perspective that combines the attributes of the Process Mining perspectives is proposed.

ACKNOWLEDGMENT

Thanks to the Consejo Nacional de Ciencia y Tecnología (CONACYT) for the financial support provided to the first author with CVU No. 634421.

REFERENCES

[1] W. M. Van der Aalst, "Extracting event data from databases to unleash process mining," In BPM-Driving innovation in a digital world, Springer, pp. 105-128, 2015.

[2] M. Ghasemi and D. Amyot, "From event logs to goals: a systematic literature review of goal-oriented process mining," Requirements Engineering, 25(1), 67-93. J. Clerk Maxwell, A Treatise on Electricity and Magnetism, 3rd ed., vol. 2. Oxford: Clarendon, 1892, pp. 68-73, 2020.

[3] W. M. Van der Aalst, "Data science in action. In Process mining," 2016, Springer, Berlin, Heidelberg.

[4] R. Kneuper, "Selected Current Trends in Software Processes," In Software Processes and Life Cycle Models, pp. 303-326, 2018, Springer, Cham.

[5] R. Zhu, Y. Dai, T. Li, Z. Ma, M. Zheng, Y. Tang, and Y. Huang, "Automatic real-time mining software process activities from SVN logs using a naive Bayes classifier," IEEE Access, 7, 2019.

[6] J. Carmona and J. Cortadella, "Process discovery algorithms using numerical abstract domains," IEEE Trans. Knowl. Data Eng., vol. 26, no. 12, pp. 3064–3076, Dec. 2014.

[7] S. J. Urrea-Contreras, B. L. Flores-Rios, M. A. Astorga-Vargas and J. E. Ibarra-Esquer, "Process Mining Perspectives in Software Engineering: A Systematic Literature Review," In 2021 Mexican International Conference on Computer Science (ENC), pp. 1-8, 2021, IEEE.

[8] R. LeBlanc, A. Sobel, J. L. Diaz-Herrera and T. B. Hilburn, "Software Engineering 2014-Curriculum Guidelines for Undergraduate Degree Programs in Software Engineering a Volume of the Computing Curricula Series," Joint Task Force on Computing Curricula IEEE Computer Society Association for Computing Machinery, 2014.

[9] B. Keith and V. Vega, "Process mining applications in software engineering," Trends and Applications in Software Engineering, Advances in Intelligent Systems and Computing, pp. 47-56, 2016, doi: 10.1007/978-3-319-48523-2_5.

[10] V. Saini, P. Singh and A. Sureka, "Control-flow based anomaly detection in the bug-fixing process of open-source projects," In Proceedings of the 13th Innovations in Software Engineering Conference on Formerly known as India Software Engineering Conference, pp. 1-11, 2020.

[11] R. Marques, M. M. da Silva and D. R. Ferreira, "Assessing agile software development processes with process mining: A case study," In 2018 IEEE 20th Conference on Business Informatics (CBI), Vol. 1, pp. 109-118, 2018, IEEE.

[12] M. L. V. Eck, X. Lu, S. J. Leemans, and W. M. Van Der Aalst, "PM2: a process mining project methodology," International conference on advanced information systems engineering. Springer, Cham. pp. 297-313, 2015.

[13] M. Macak, D. Kruzelova, S. Chren and B. Buhnova, "Using process mining for Git log analysis of projects in a software development course," Education and Information Technologies, 26(5), pp. 5939-5969, 2021.

[14] E. Kalliamvakou, G. Gousios, K. Blincoe, L. Singer, D. M. German and D. Damian, "An in-depth study of the promises and perils of mining GitHub," Empirical Software Engineering, 21(5), pp. 2035-2071, 2016.

[15] P. Ardimento, M. L. Bernardi, M. Cimitile and F. M. Maggi, "Evaluating coding behavior in software development processes: A process mining approach," In 2019 IEEE/ACM International Conference on Software and System Processes (ICSSP), pp. 84-93, 2019, IEEE.

[16] J. Caldeira, F. B. e Abreu, J. Reis and J. Cardoso, "Evaluating Assessing Software Development Teams' Efficiency using Process Mining," In 2019 International Conference on Process Mining (ICPM), pp. 65-72, 2019, IEEE.

[17] Scrum-institute, Scrum revealed training book (3rd ed.), International Scrum Institute, 2017. https://bit.ly/3rKMF31

[18] E. Brechner, "Agile project management with Kanban," Pearson Education, 2015.

[19] Atlassian, "Jira Software," n.d. Recuperado de https://www.atlassian.com/software/jira

[20] SaaSJe, "Issue History for Jira," n.d. Recuperado de thttps://marketplace.atlassian.com/apps/1220385/issue-history?tab=overview&hosting=cloud

[21] GitHub, "GitHub," n.d. Recuperado de https://github.com/

[22] J. D. H. Reséndiz, E. T. Leal, H. M. M. Castro, and G. R. Galván, "Application of unsupervised learning techniques for clustering traces in the Process Mining domain," Pistas Educativas, 2019.

[23] J. Carmona, B. van Dongen, A. Solti, and M. Weidlich, "Conformance checking," 2018, Switzerland: Springer.

[24] A. Berti and W. M. Van der Aalst, "A novel token-based replay technique to speed up conformance checking and process enhancement," Transactions on Petri Nets and Other Models of Concurrency, Springer, pp. 1-26, 2021.

[25] J. Yang, C. Ouyang, W. M. Van der Aalst, A. H. ter Hofstede and Y. Yu, "OrdinoR: A framework for discovering, evaluating, and analyzing organizational models using event logs," Decision Support Systems, 2022.

[26] M. Gupta, A. Sureka and S. Padmanabhuni, "Process mining multiple repositories for software defect resolution from control and organizational perspective," In Proceedings of the 11th Working Conference on Mining Software Repositories, pp. 122-131, 2014.

[27] F. Mannhardt, "Multi-perspective Process Mining," In BPM (Dissertation/Demos/Industry), pp. 41-45, 2018.

[28] TFPM – IEEE Task Force on Process Mining, "Process Mining Manifesto," Business Process Management Workshops, BPM 2011, Lecture Notes in Business Information Processing, vol. 99, pp. 169-194, 2012, doi: 10.1007/978-3-642-28108-2_19

Authorized licensed use limited to: Pontifica Universidade Catolica do Parana. Downloaded on April 12,2026 at 23:15:57 UTC from IEEE Xplore. Restrictions apply.